

From the output, it can be seen that there are `.text`, `.data`, `.bss`, `.symtab` and `.shstrtab` sections. To check the code that is present in `.text`, use the following command:

```
objdump -d study
```

This will dump all the code that is present in our `study.c` example. Try experimenting with `objdump`, `hd` and `readelf` commands to analyse each section of the ELF file. Modify the program, compile it and see how different information in the section changes.

So how does the compiler come to know where to put each section in the final ELF. This is where the linker comes in. Linkers use a file known as the linker descriptor file. This contains information about all the memory in the target machine (Flash, RAM, QSPI, SDRAM, etc) with its starting address and size. It also contains information about the different sections that should be present in the final ELF file and where each section should be loaded in the target machine. The following is a simple linker descriptor file:

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x0, LENGTH =
0x020

    RAM (rwx) : ORIGIN = 0xA0, LENGTH =
0x00A0000 /* 640K */
}

ENTRY(Reset_Handler)
```

SECTIONS

```
{
    . = 0x0000;
    .text : { *(.text) }
    . = 0xA0;
    .data : { *(.data) }
    .bss : { *(.bss) }
}
```

The above linker script file informs

linker about the following:

- The system has two memories — Flash and RAM.
- Put all executable code that is present in the `.text` section at the address 0x00000 in the target machine.
- Put all data (`.data` and `.bss`) starting at address 0xA0.

ELF program header

The program header in the ELF specifies which section of the ELF file goes to a particular memory location. The program header location in the ELF file is provided by the `e_phoff` variable present in the ELF header. `e_phoff` provides the offset at which the ELF program header is present in the ELF file. `e_phnum` is the number of program header entries in the ELF file and the size of each entry is `e_phentsize`.

Use the following command to see the program header information of our `study.c` example:

```
readelf -l study
```

Linker

This is the final stage of compilation. In this stage, the final executable ELF is generated. Linker takes all object files generated for each programming file in previous stages as inputs. It also takes a linker descriptor file as input. Linker combines sections from all object files into one final ELF. It instructs

the linker descriptor file as to what addresses are to be provided for each section. It also resolves the symbols used by each object file. If any symbol is not defined, then linker gives an error. These are the stages by which a final executable is generated.

In the case of ARM and other microcontroller platforms, Hex files are present. These are generated from ELF files using the following command:

```
objcopy -O ihex study study.hex
```

Program execution

If you execute any C program, then `main()` is always the entry point. This is because a startup code is always linked to your original program by linker. This startup code consists of a startup function, which usually is `__start` or `Reset_Handler`. This is placed at the address that the processor will execute at boot. This function calls the `main()` function, which is the starting point of the application. Generally, in embedded systems, there is a `startup.s` file which contains a vector table where the `Reset_Handler` function is placed, which the controller executes at startup. This `Reset_Handler` will copy all initialised variables to RAM, and then initialise all the memory used by the BSS section to zero. This `Reset_Handler` will then call the `main()` function. This is how an application is compiled and executed. **END** 

References

- [1] <https://www.calleerlandsson.com/the-four-stages-of-compiling-a-c-program/>
- [2] https://en.wikipedia.org/wiki/GNU_Binutils
- [3] <https://cirosantilli.com/elf-hello-world>
- [4] http://www.skyfree.org/linux/references/ELF_Format.pdf
- [5] <https://docs.huihoo.com/redhat/rhel-4-docs/rhel-ld-en-4/simple-example.html>

By: Raghavendra Chandra Ganiga

The author is a senior embedded firmware development engineer at Enlite Research Pvt Ltd, Mumbai. His interests are microcontroller firmware, sensors, RTOS development and Linux device drivers.